

RUHR-UNIVERSITÄT BOCHUM

Fundamentals of GPU Programming
Lecture 2

Denis Eremin

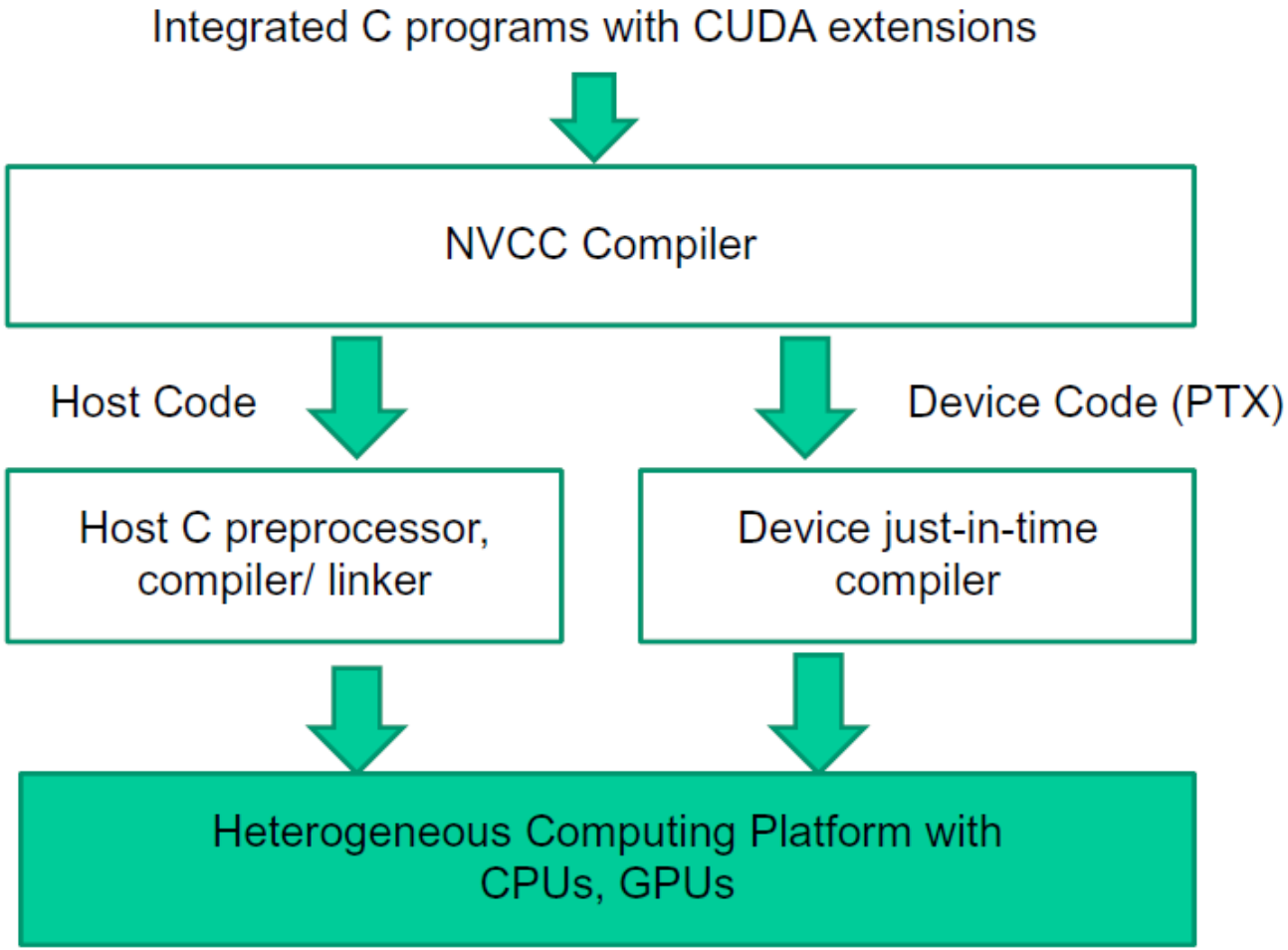
Fakultät für Elektrotechnik und Informationstechnik
Lehrstuhl für Theoretische Elektrotechnik

Thread Hierarchy

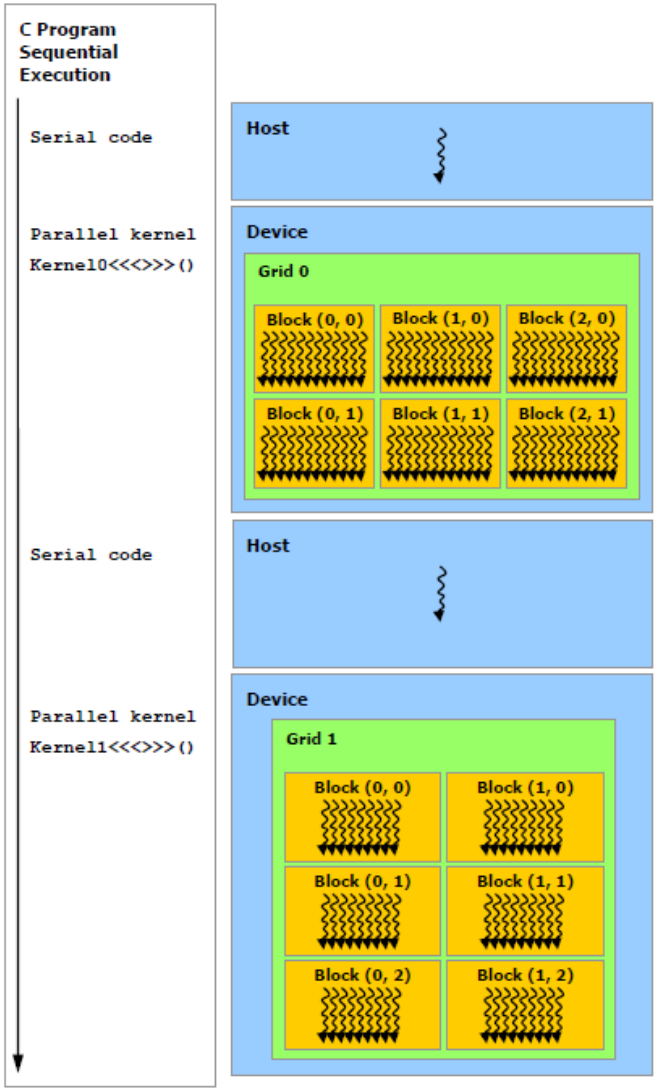
We will learn

- Simple example: vector addition
- Memory operations on device
- Function declaration on device
- Kernel launch, execution configuration
- Thread identification

Compilation of a CUDA Program



Execution of a CUDA Program



Host = CPU

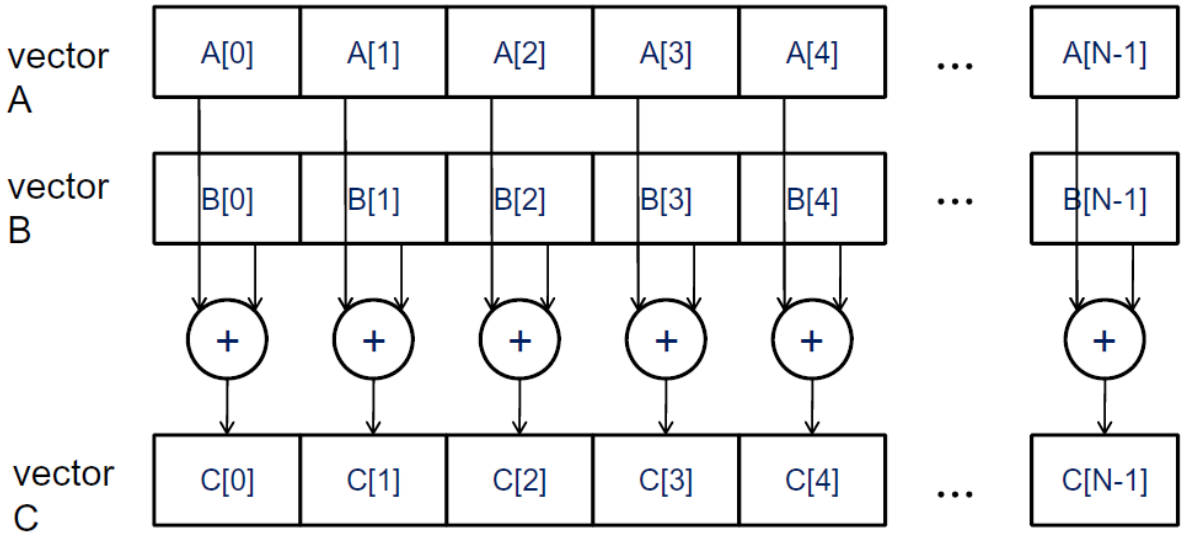
Device = GPU

Kernel = Function executed on Device

Grid = All threads generated by Kernel

Note: execution on GPU is asynchronous, i.e. independent of CPU unless a barrier is invoked!

Vector Addition: Data Parallelism



Can be also used in parallelization: task parallelism and instruction parallelism

Vector Addition: Serial Version

```
// compute vector sum h_C = h_A + h_B
void vecAdd(float* h_A, float* h_B, float* h_C, int n)
{
  for (i=0; i<n; i++) h_C[i] = h_A[i] + h_B[i];
}

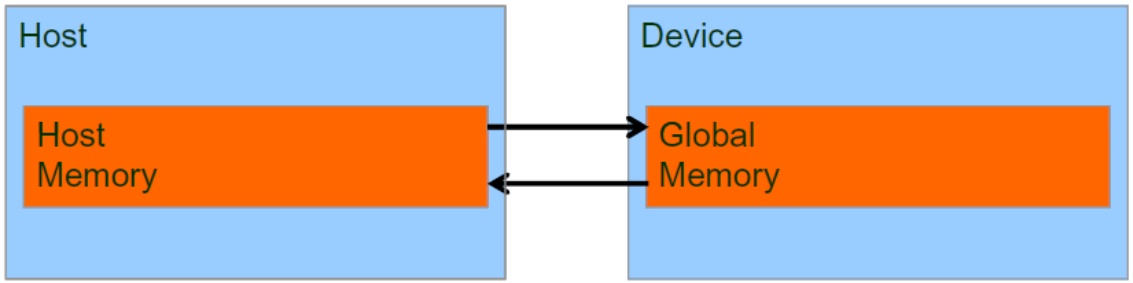
int main()
{
  // memory allocation for h_A, h_B, and h_C
  // i/o to read h_A and h_B, N elements each
  ...
  vecAdd(h_A, h_B, h_C, N);
}
```

Structure of the Parallel Version

```
#include <cuda.h>
...
void vecAdd(float* A, float* B, float* C, int n)
{
  int size = n*sizeof(float);
  float *A_d, *B_d, *C_d;
  ...
1.// allocate device memory for A, B, and C
  // copy A and B to device memory

2.// kernel launch code - to have the device
  // to perform the actual vector addition

3.// copy C from the device memory
  // free device vectors
}
```



Memory Functions

STANDARD C FUNCTIONS	CUDA C FUNCTIONS
malloc	cudaMalloc
memcpy	cudaMemcpy
memset	cudaMemset
free	cudaFree

cudaMalloc and cudaFree Syntax

```
cudaError_t cudaMalloc (void ** devPtr, size_t size)

Parameters:
 devPtr  - Pointer to allocated device memory
 size  - Requested allocation size in bytes

Returns:
cudaSuccess, cudaErrorMemoryAllocation

cudaError_t cudaFree  ( void *  devPtr   )

Parameters:
 devPtr  - Device pointer to memory to free

Returns:
cudaSuccess, cudaErrorInvalidDevicePointer, cudaErrorInitializationError

Note:
Note that this function may also return error codes from previous, asynchronous launches.
```

CUDA Device Memory Allocation: Example

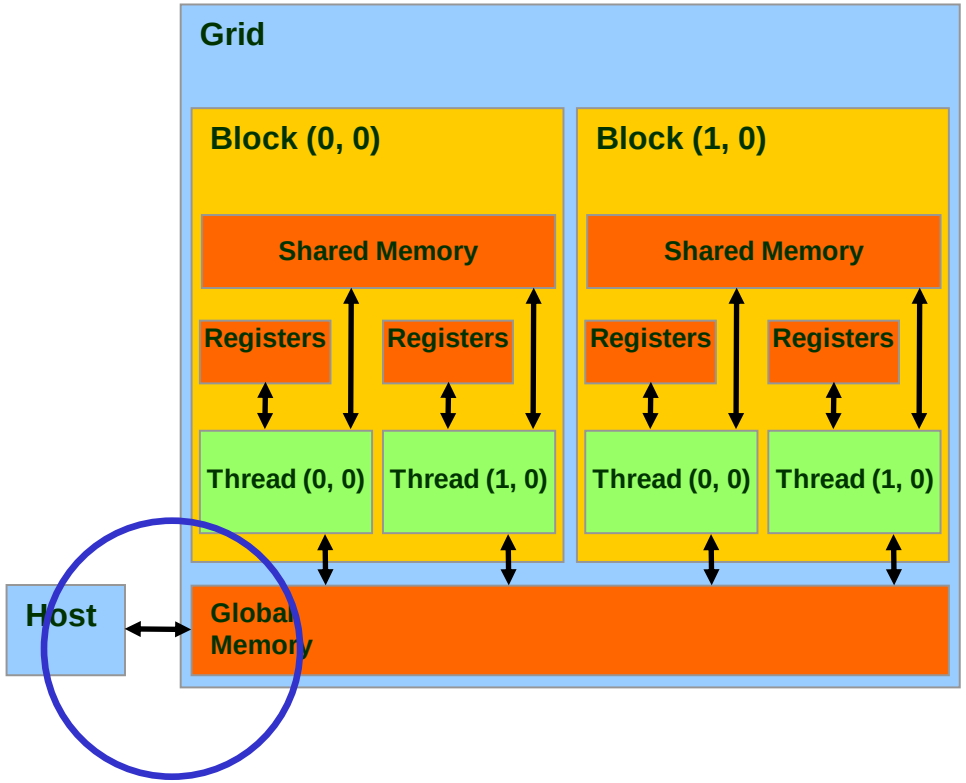
Code example:

- Allocate a n single precision float vector
- Attach the allocated storage to d_A
- Free the storage after work is done
- “d” is often used to indicate a device data structure

```
float *d_A;
int size = n*sizeof(float);
cudaMalloc((void**)&d_A, size);
...
cudaFree(d_A);
```

CUDA Host-Device Data Transfer

- **cudaMemcpy()**
  - memory data transfer
  - Requires four parameters
    - Pointer to destination
    - Pointer to source
    - Number of bytes copied
    - Type of transfer
      - Host to Host
      - Host to Device
      - Device to Host
      - Device to Device
- Asynchronous transfer ?



Cannot be used for the memory transfer between multiple GPUs!

cudaMemcpy Syntax

```
cudaError_t cudaMemcpy (void *dst, const void *src, size_t count, enum cudaMemcpyKind kind)

Copies count bytes from the memory area pointed to by src
to the memory area pointed to by dst, where kind is one of
cudaMemcpyHostToHost, cudaMemcpyHostToDevice,
cudaMemcpyDeviceToHost, or cudaMemcpyDeviceToDevice,
and specifies the direction of the copy. The memory areas
may not overlap. Calling cudaMemcpy() with dst and src pointers
that do not match the direction of the copy results in
an undefined behavior.

Parameters:
 dst  - Destination memory address
 src  - Source memory address
 count  - Size in bytes to copy
 kind  - Type of transfer

Returns:
cudaSuccess, cudaErrorInvalidValue, cudaErrorInvalidDevicePointer,
cudaErrorInvalidMemcpyDirection
```

Parallel Version contd.

```
void vecAdd(float* A, float* B, float* C, int n)
{
  int size = n*sizeof(float);
  float *d_A, *d_B, *d_C;

  cudaMalloc((void**)&d_A, size);
  cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
  cudaMalloc((void**)&d_B, size);
  cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);

  cudaMalloc((void**)&d_C, size);

  // kernel invocation code - to be shown later
  ...

  cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);

  // free device memory for A, B, C
  cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

Kernels

Kernel is a function to be executed on Device

kernel function specifier

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
}
```

kernel declaration

Warning: only valid for a single block!

kernel launch

execution configuration

Each thread is accessible through a unique $0 \leq \text{threadIdx} \leq N - 1$

$N \leq 1024$ (kernel does not launch if violated)

Built-in Preinitialized Thread Coordinate System

- Each thread uses IDs to decide what data to work on
  - Block ID: 1D, 2D, or 3D
  - Thread ID: 1D, 2D, or 3D

- Simplifies memory addressing when processing multidimensional data

Coordinates

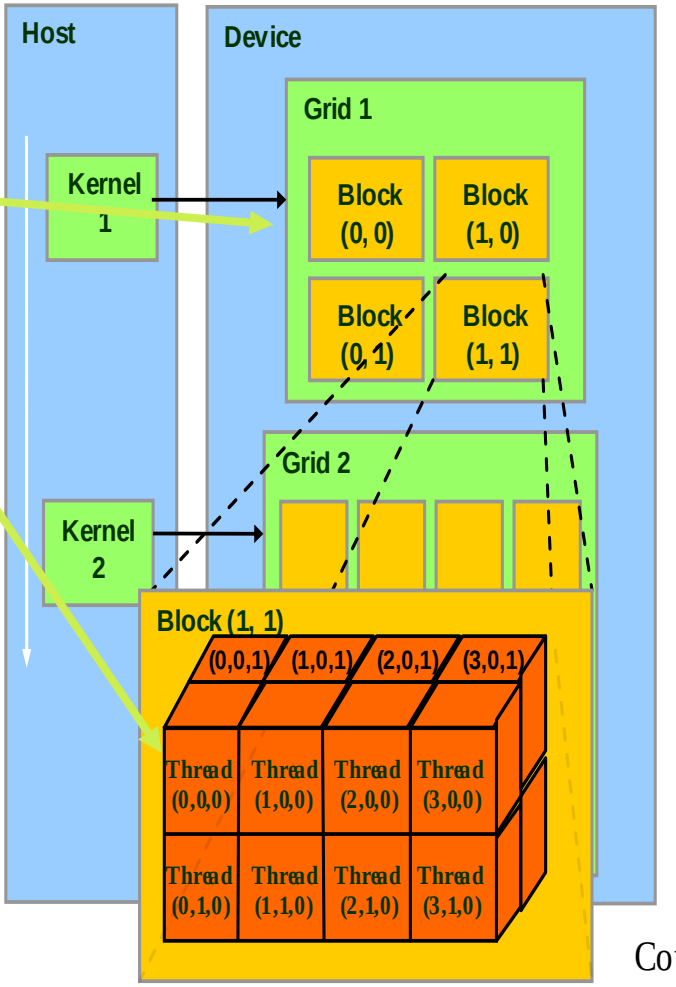
$\mathrm{dim3\ threadIdx,\ blockIdx}$

Next-level sizes

$\mathrm{dim3\ blockDim,\ gridDim}$

$$0 \leq \mathrm{threadIdx.x} \leq \mathrm{blockDim.x} - 1$$

$$0 \leq \mathrm{blockIdx.x} \leq \mathrm{gridDim.x} - 1$$



Courtesy: NDVIA

Example

```
#include <cuda_runtime.h>
#include <stdio.h>

__global__ void checkIndex(void) {
    printf("threadIdx:(%d, %d, %d)  blockIdx:(%d, %d, %d)  blockDim:(%d, %d, %d) "
        "gridDim:(%d, %d, %d)\n", threadIdx.x, threadIdx.y, threadIdx.z,
        blockIdx.x, blockIdx.y, blockIdx.z, blockDim.x, blockDim.y, blockDim.z,
        gridDim.x,gridDim.y,gridDim.z);
}

int main(int argc, char **argv) {
   // define total data element
   int nElem = 6;

   // define grid and block structure
   dim3 block (3);
   dim3 grid  ((nElem+block.x-1)/block.x);

   // check grid and block dimension from host side
   printf("grid.x %d grid.y %d grid.z %d\n",grid.x, grid.y, grid.z);
   printf("block.x %d block.y %d block.z %d\n",block.x, block.y, block.z);

   // check grid and block dimension from device side
   checkIndex <<<grid, block>>> ();

   // reset device before you leave
   cudaDeviceReset();

   return(0);
}
```

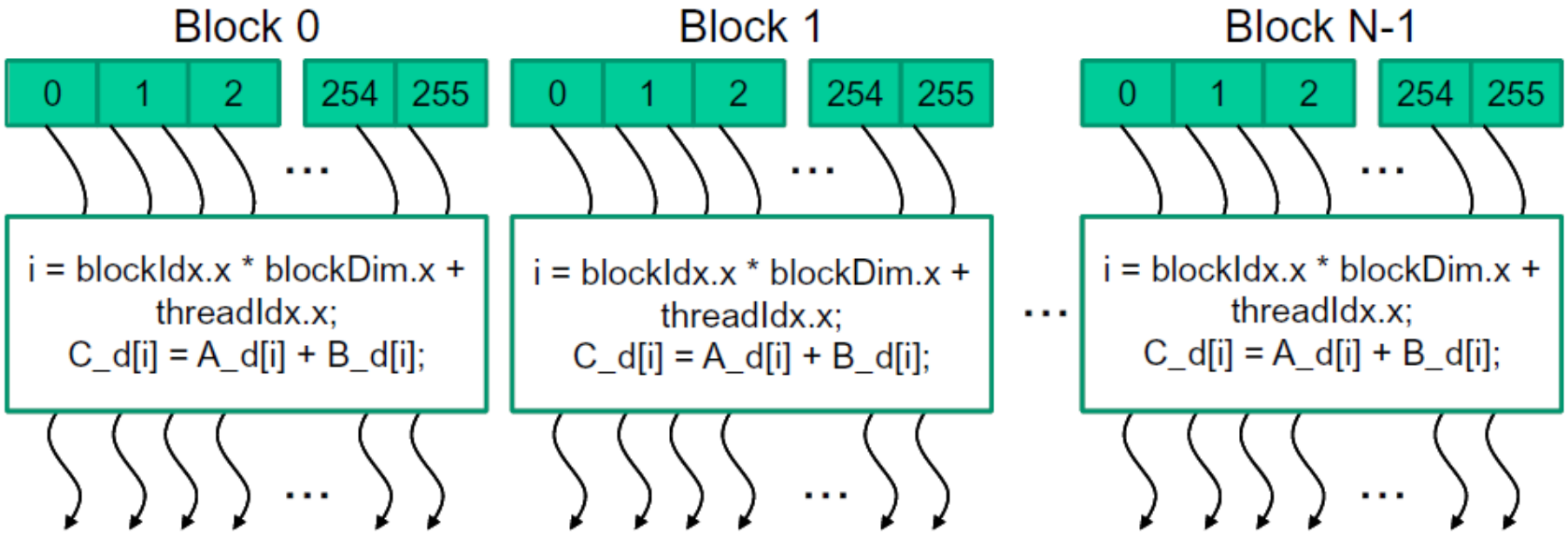
Compile and run the code:

```
$ nvcc -arch=sm_20 checkDimension.cu -o check
$ ./check
```

Output:

```
grid.x 2 grid.y 1 grid.z 1
block.x 3 block.y 1 block.z 1
threadIdx:(0, 0, 0)  blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(1, 0, 0)  blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(2, 0, 0)  blockIdx:(1, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(0, 0, 0)  blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(1, 0, 0)  blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
threadIdx:(2, 0, 0)  blockIdx:(0, 0, 0) blockDim:(3, 1, 1) gridDim:(2, 1, 1)
```

Example: 1D Grid of 1D Blocks




```
dim3 dimGrid(N,1,1);
dim3 dimBlock(M,1,1);
kernelFunction <<< dimGrid, dimBlock >>> (...);
```

or

```
kernelFunction <<< N, M >>> (...);
```

Vector Addition: Improved Kernel

```
// compute vector sum C = A + B
// each thread performs one pairwise addition
__global__ void vecAddKernel(float* A, float* B, float* C, const int N)
{
  int idx = threadIdx.x + blockDim.x*blockIdx.x;
  if (idx<N) C[idx] = A[idx] + B[idx];
}
```

CUDA C keywords for function declaration

	Executed on the:	Only callable from the:
`__device__ float DeviceFunc()`	device	device
`__global__ void KernelFunc()`	device	host
`__host__ float HostFunc()`	host	host

Vector Addition: Complete Version of the Wrapper

```
void vecAdd(float *A, float* B, float *C, int n)
{
  int size = n*sizeof(float);
  float *d_A, *d_B, *d_C;

  cudaMalloc((void**)&d_A, size);
  cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
  cudaMalloc((void**)&d_B, size);
  cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);
  cudaMalloc((void**)&d_C, size);

  vecAddKernel <<< ceil(n/256.0), 256 >>> (d_A, d_B, d_C, n);

  cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);

  cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

Full Study of the Vector Addition Example

RUB

Defensive programming: write test first:

```
void checkResult(float *hostRef, float *gpuRef, const int N) {
   double epsilon = 1.0E-8;
   bool match = 1;
   for (int i=0; i<N; i++) {
      if (abs(hostRef[i] - gpuRef[i]) > epsilon) {
         match = 0;
         printf("Arrays do not match!\n");
         printf("host %5.2f gpu %5.2f at current %d\n",hostRef[i],gpuRef[i],i);
         break;
      }
   }

   if (match) printf("Arrays match.\n\n");
}
```

Initialize arrays with random numbers:

```
void initialData(float *ip,int size) {
   // generate different seed for random number
   time_t t;
   srand((unsigned) time(&t));

   for (int i=0; i<size; i++) {
      ip[i] = (float)( rand() & 0xFF )/10.0f;
   }
}
```

host version:

```
void sumArraysOnCPU(float* A, float* B, float* C, const int N) {
  for (int idx=0; idx<N; idx++) C[idx] = A[idx] + B[idx];
}
```

device version:

```
__global__ void sumArraysOnGPU(float* A, float* B, float* C, const int N) {
  int idx = threadIdx.x + blockIdx.x*blockDim.x;
  if (idx<N) C[idx] = A[idx] + B[idx];
}
```

The “for” loop in the CPU version is replaced by the grid of threads in the GPU version!

Timing Kernel Execution

Using a CPU timer:

```
double cpuSecond() {
   struct timeval tp;
   gettimeofday(&tp,NULL);
   return ((double)tp.tv_sec + (double)tp.tv_usec*1.e-6);
}
```

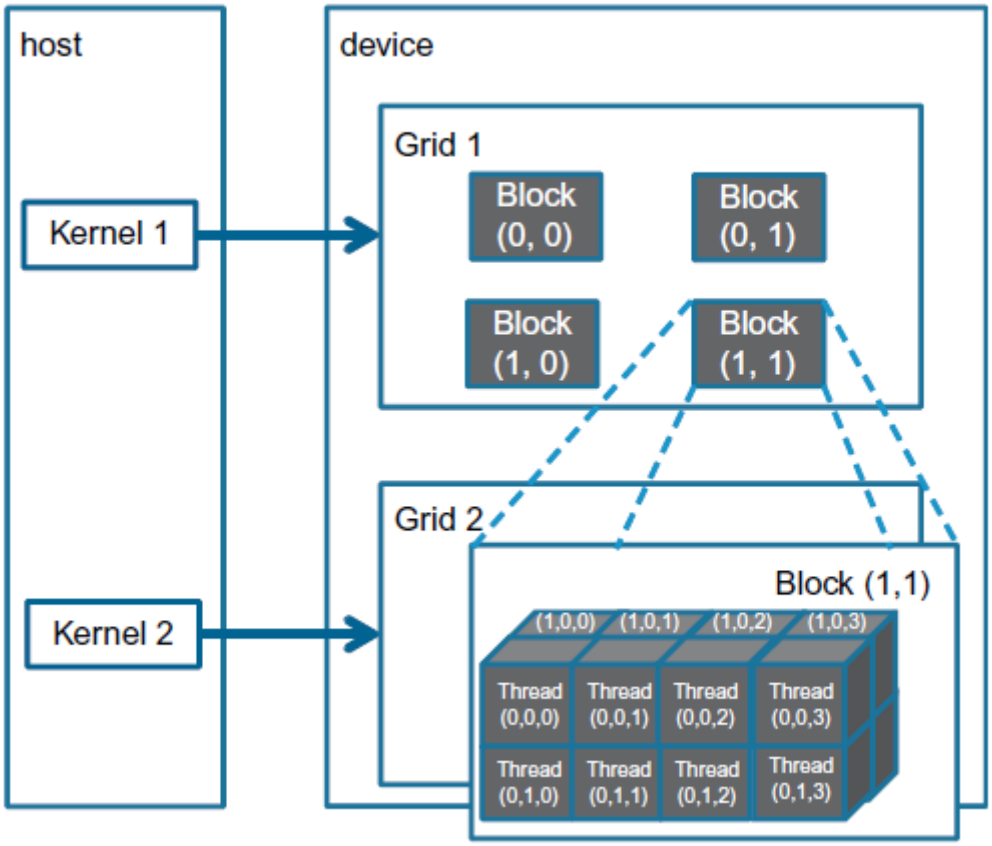
```
double iStart = cpuSecond();
kernel_name<<<grid, block>>>(argument list);
cudaDeviceSynchronize();
double iElaps = cpuSecond() - iStart;
```

Using the nvprof profiler:

```
$ nvprof ./sumArraysOnGPU-timer
```

```
==17770== Profiling application: ./sumArraysOnGPU-timer
==17770== Profiling result:
Time(%)      Time     Calls       Avg       Min       Max  Name
 70.35%  52.667ms         3  17.556ms  17.415ms  17.800ms  [CUDA memcpy HtoD]
 25.77%  19.291ms         1  19.291ms  19.291ms  19.291ms  [CUDA memcpy DtoH]
  3.88%  2.9024ms         1  2.9024ms  2.9024ms  2.9024ms  sumArraysOnGPU
(float*, float*, int)
```

Example: Multidimensional Grid




```
dim3 dimBlock(2,2,1);
dim3 dimGrid(4,2,2);
KernelFunction <<< dimGrid, dimBlock >>> (...);
```

Specifying the Execution Configuration

Execution configuration fixed at compile time

```
dim3 dimBlock(128,1,1);
dim3 dimGrid(32,1,1);
vecAddKernel <<< dimGrid, dimBlock >>> (...);
```

Execution configuration can be changed during the run time

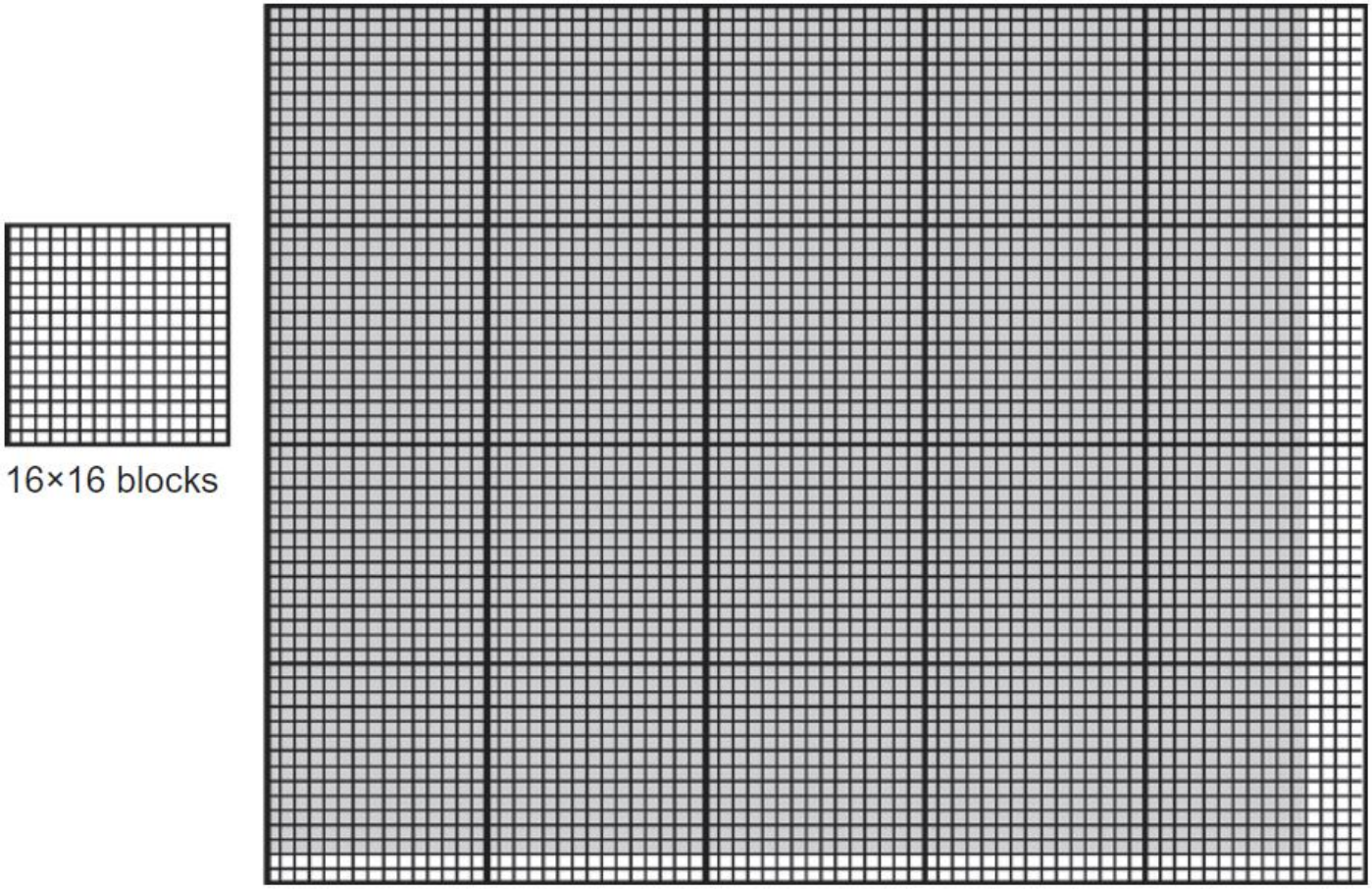
```
dim3 dimGrid(ceil(n/256.0),1,1);
dim3 dimBlock(256,1,1);
vecAddKernel <<< dimGrid, dimBlock >>> (...);
```

Limiting Factors: max total number of threads per block 1024

max number of threads 1024 x 1024 x 64

max number of blocks 2^31 in x, 2^16 in y and z

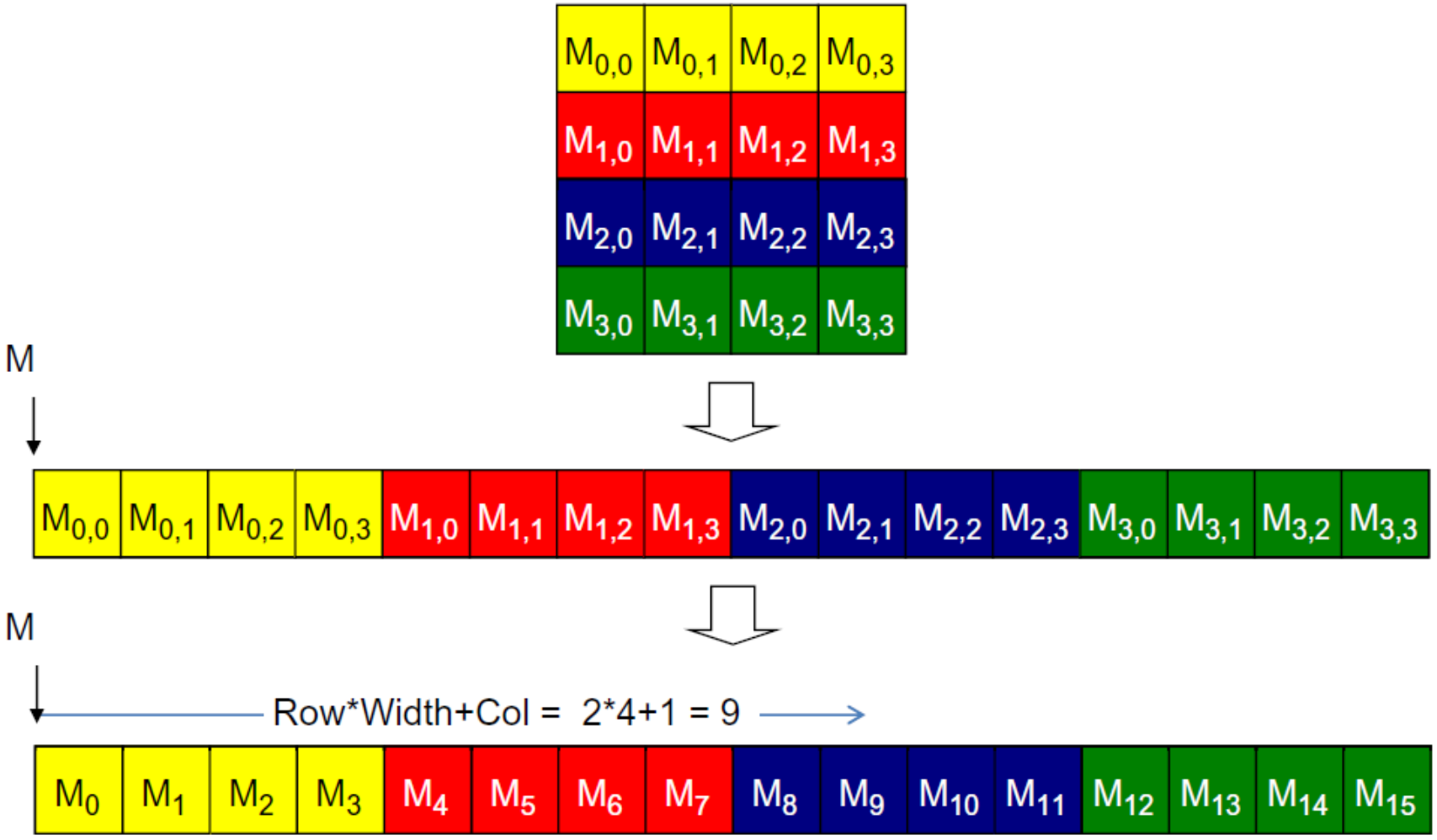
Example: Processing 2D Picture



Problem: need to process 76 x 62 picture

Processing Grid: 80 x 64

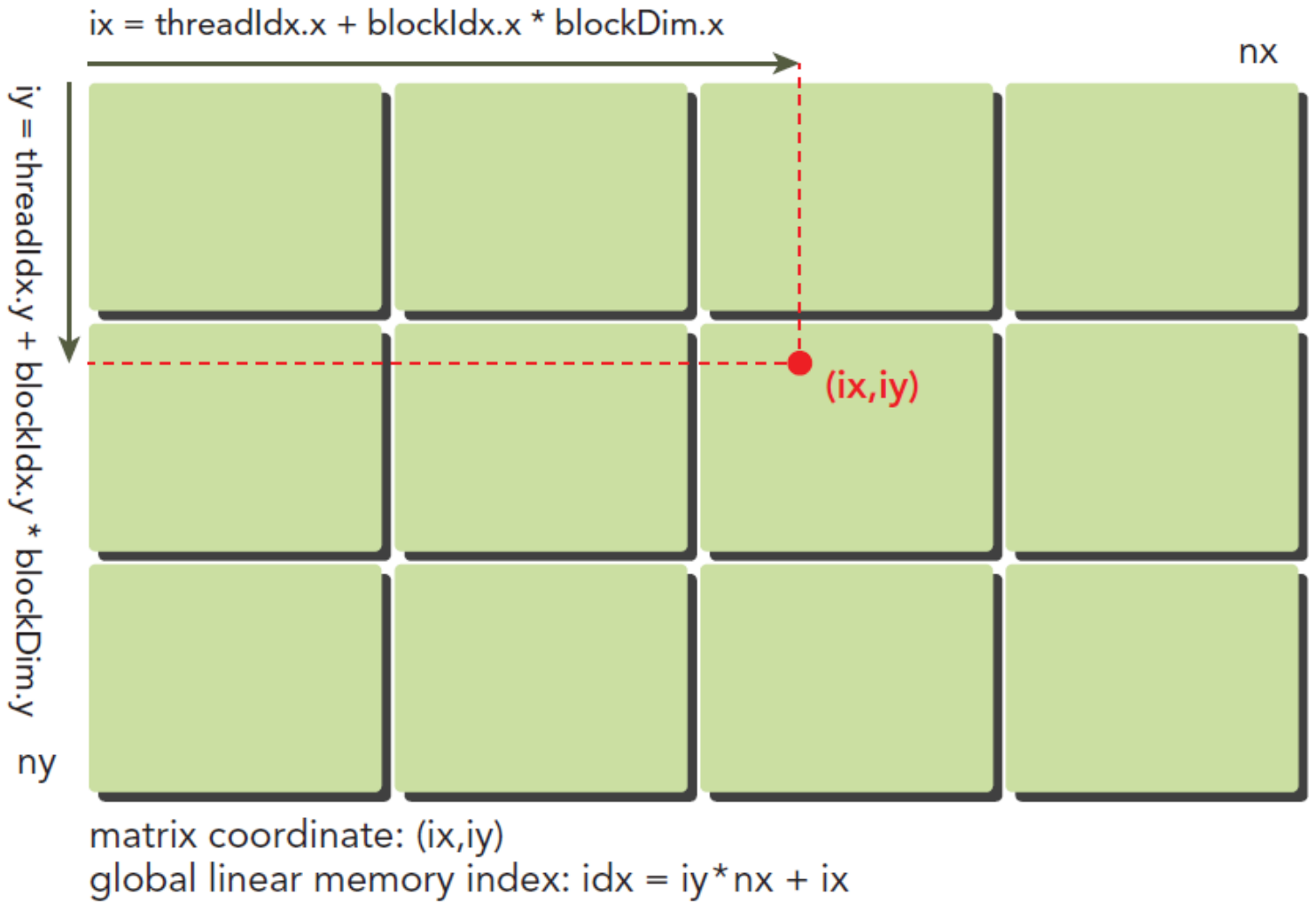
Memory Layout of a Matrix in C (Index Flattening)



coordinate of a given point in the matrix:

```
ix = threadIdx.x + blockIdx.x * blockDim.x
iy = threadIdx.y + blockIdx.y * blockDim.y
```

offset in linear global memory: `idx = iy * nx + ix`



Kernel:

```
__global__ void PictureKernel(float* d_Pin, float* d_Pout, int n, int m)  {

  // calculate the row# of the d_Pint and d_Pout element to process
  int row = blockIdx.y*blockDim.y + threadIdx.y;

  // calculate the column# of the d_Pint and d_Pout element to process
  int col = blockIdx.x*blockDim.x + threadIdx.x;

  // each thread computes one element of d_Pout if in range
  if ((row<m) && (col<n)) {
    d_Pout[row*n+col] = 2*d_Pin[row*n+col];
  }

}
```

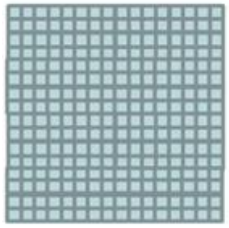
Execution configuration:

alternatively, use

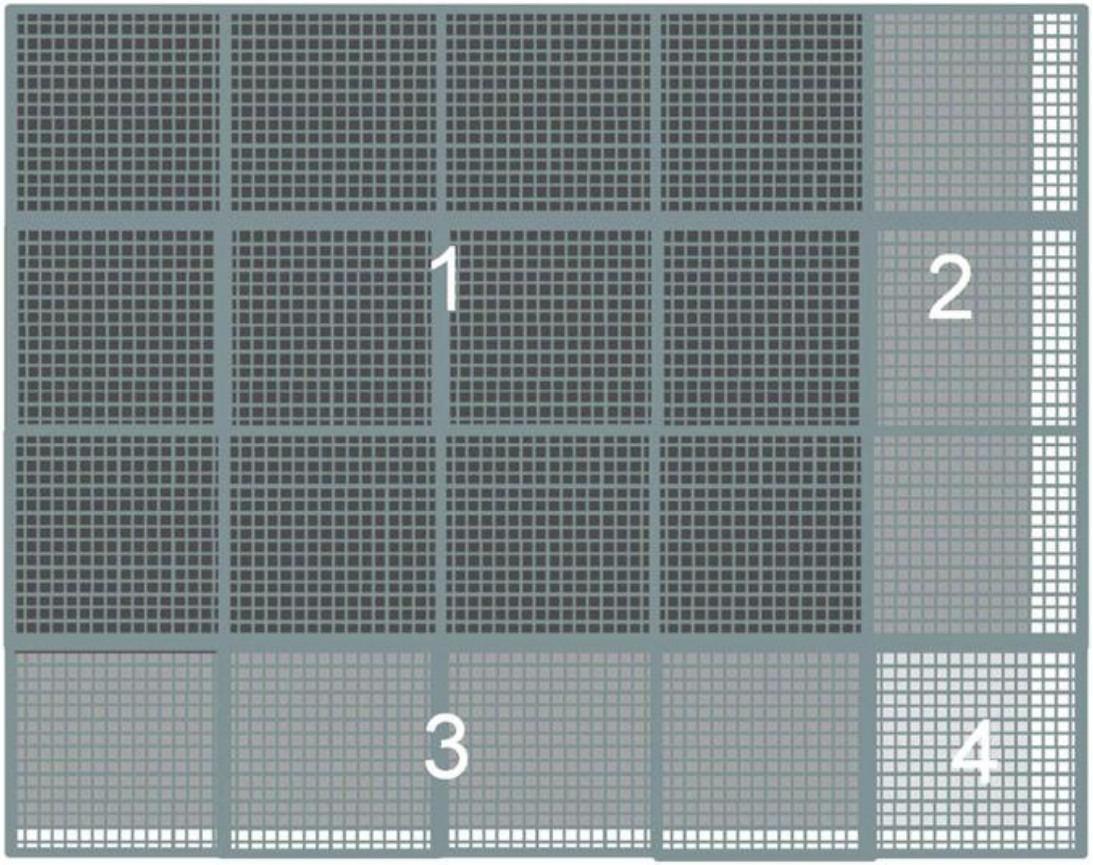
```
dim3 dimGrid((n+16-1)/16, (m+16-1)/16, 1);
```

```
dim3 dimBlock(16,16,1);
dim3 dimGrid(ceil(n/16.0), ceil(m/16.0), 1);
PictureKernel <<< dimGrid, dimBlock >>> (d_Pin, d_Pout, n, m);
```

The Processing Grid



16x16 block



76 x 62 picture

80 x 64 grid

Example: Matrix Addition

KERNEL CONFIGURATION	KERNEL ELAPSED TIME	BLOCK NUMBER
(32,32)	0.060323 sec	512 x 512
(32,16)	0.038041 sec	512 x 1024
(16,16)	0.045535 sec	1024 x 1024

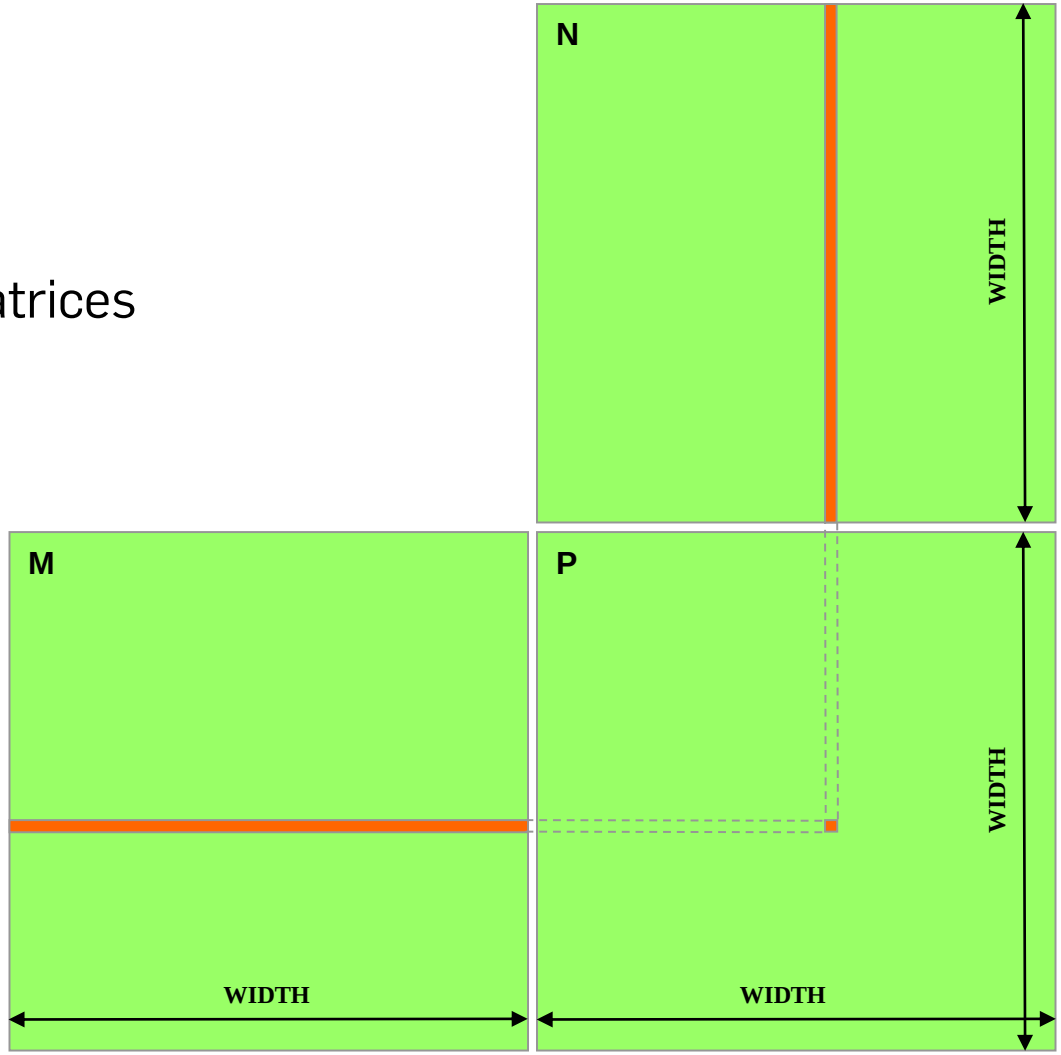
Model Problem: Square Matrix Multiplication

$$\mathbf{P} = \mathbf{M} \cdot \mathbf{N} \qquad P_{ij} = \sum_k M_{ik} N_{kj}$$

$(\mathbf{M}, \mathbf{N}) = \mathrm{WIDTH} \times \mathrm{WIDTH}$ matrices

Data-parallel algorithm:

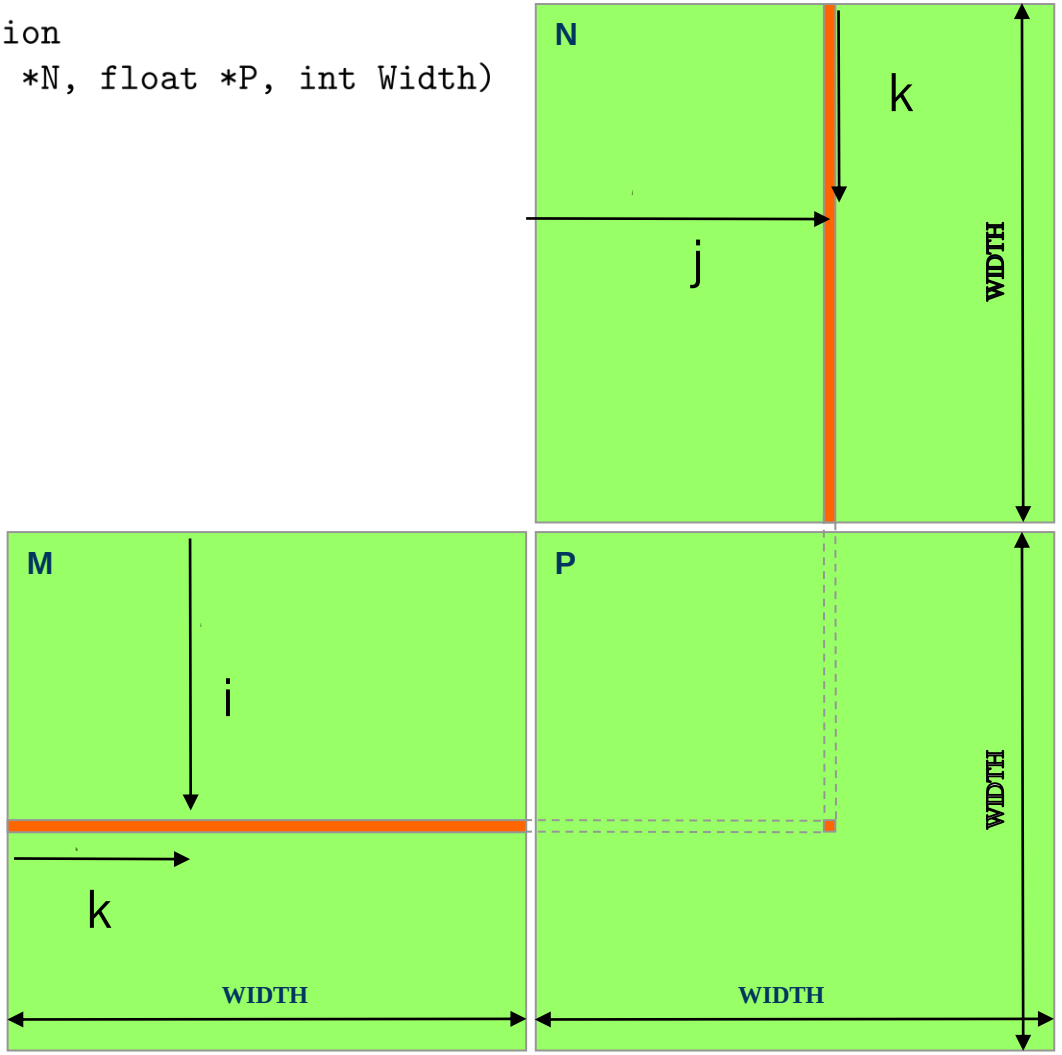
- one thread calculates one element of P
- M and N are loaded WIDTH times from global memory



Matrix Multiplication Function (Host)

```
// matrix multiplication function definition
void MatrixMultiplication(float *M, float *N, float *P, int Width)
{
  for (int i=0; i<Width; i++)
  {
    for (int j=0; j<Width; j++)
    {
      float sum = 0;
      for (int k=0; k<Width; k++)
      {
        float a = M[i*Width + k];
        float b = N[k*Width + j];
        sum += a*b;
      }
      P[i*Width + j] = sum;
    }
  }

}
```



Parallel Version: General Code Structure

```
// matrix multiplication function declaration
void MatrixMultiplication(float *M, float *N, float *P, int Width)


// main programm
int main(void)
{
  1. // Allocate and initialize the matrices M, N, P
     ...

  2. // M*N algorithm
     MatrixMultiplication(M, N, P, Width);

  3. // Free memory occupied by M, N, P
     ...
}

// matrix multiplication function definition
void MatrixMultiplication(float *M, float *N, float *P, int Width)
{
     ...
}
```

Matrix Multiplication Function (Device)

```
// matrix multiplication function definition
void MatrixMultiplication(float *M, float *N, float *P, int Width)
{
   1. // allocate M, N, P on device
   ...

   2. // initialize M, N, P on device (transfer from host to device)
   ...

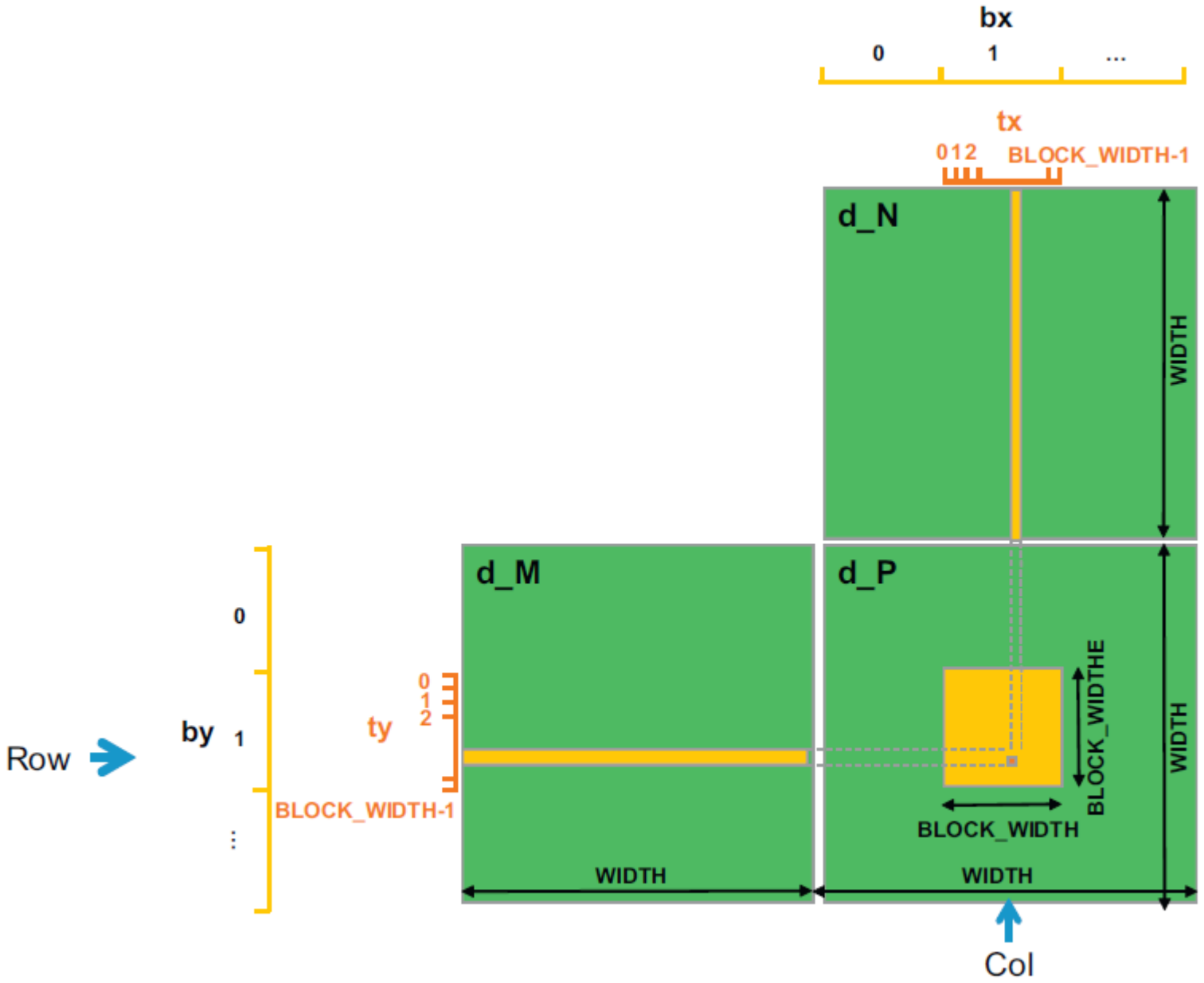
   3. // launch the matrix multiplication kernel
   MatrixMultiplicationKernel <<< numBlocks, numThreads >>> (M, N, P, Width);

   4. // transfer M, N, P from device to host
   ...

   5. // free memory on the device
   ...

}
```

Matrix Multiplication: Tiled Algorithm



Tiled Algorithm Kernel

```
__global__ void MaxtrixMulKernel(float* d_M, float* d_N, float* d_P, int Width) {

  // calculate the row index of the d_P and d_M
  int row = blockIdx.y*blockDim.y + threadIdx.y;

  // calculate the column index of d_P and d_N
  int col = blockIdx.x*blockDim.x + threadIdx.x;

  if ((row<Width) && (col<Width)) {
    float pvalue = 0;
    // each thread computes one element of the block sub-matrix
    for (int k=0; k<Width; k++) pvalue += d_M[row*Width+k]*d_N[k*Width+col];
    d_P[row*Width+col] = pvalue;
  }

}
```

Execution Configuration

```
#define BLOCK_WIDTH 16

...

// set the execution configuration
int numBlocks = ceil(Width/float(BLOCK_WIDTH));
dim3 dimGrid(numBlocks,numBlocks);
dim3 dimBlock(BLOCK_WIDTH,BLOCK_WIDTH);

// launch the kernel
MatrixMulKernel <<< dimGrid, dimBlock >>> (d_M, d_N, d_P, Width);
```

Homework Assignment #2 (4 pts), due on 29.10.26

RUB

1) Implement parallel version of the matrix addition code on GPU for matrices of arbitrary size.

2) Compare execution time (excluding the data transfers) for matrices of 10x10, 100x100, 1000x1000, 500x2000,100x10000 float random elements. Do not include the host<->device memory transfer time.

3) Compare analogous execution times for a CPU version of matrix addition

4) Compare execution time for three different execution configurations (with the blocks of 16 x 16, 16 x 32, 32 x 16 threads) for the 100 x 10000 case on GPU

Guidelines for the Homework Submission:

Make sure that each homework solution that you submit contains 4 files (any missing file will result in 1 point less):

1. The code itself. Write two versions, one for CPU and another for GPU and ALWAYS compare correctness of the data between these versions (within the appropriate precision, pay attention to that!)
2. A screenshot which shows execution of your code on your computational system
3. Results. When measuring execution time, make sure it is accurate enough(up to several digits after the comma in ms). Do not include the host<->device memory transfer time unless explicitly told to do so
4. Analysis/discussion of the results. Whenever applicable, describe what you see and propose a possible explanation based on the material that we have covered in the class. [Comment: This does not apply to the HW#2 as I still haven't told you enough to build a basis for the analysis. However, starting from HW#3 I will expect it from you.]

Upload the submission to the moodle. If your submission misses the deadline, the score will be cut by half

Work in groups is okay, but be sure to indicate your group partners at the top of the submitted code file; Each group member has to upload the full solution package to moodle. Any detected use of AI will void the submission

Guidelines for the Homework Submission 2:

Make sure that the code file in each homework submission is structured as follows:

1. If appropriate, CPU function(s) definition for a sequential analog of the program that we want to parallelize on GPU. This will serve to gauge the performance of the GPU version against, and to check the GPU output correctness.
2. GPU kernel(s) definition.
3. A CPU function definition for comparing the CPU and GPU outputs.
4. Additional functions (if needed)
5. The main program, where

   a) infrastructural tasks are performed (allocation and initialization of arrays), CPU-GPU data transfers, etc.

   b) CPU function(s) call(s), with the corresponding time measurement (if needed)

   c) GPU kernel(s) launch(es), with the corresponding time measurement

   d) Call of the function performing the comparison between CPU and GPU outputs

Note that to better measure the time it is advisable to do function and kernel call(s) in a loop, dividing the final execution time by the number of loop iterations. This will amortize the overheads.

Execution Time Measurement, Error Handling

```
...
#include  <helper_cuda.h>
#include  <helper_timer.h>


...
float tStart, tEnd;
static StopWatchInterface *timer = NULL;

sdkCreateTimer(&timer);

tStart = sdkGetTimerValue(&timer);
sdkStartTimer(&timer);

yourKernel <<< Nblocks, Nthreads >>> (a,b,c);

checkCudaErrors( cudaDeviceSynchronize() );

sdkStopTimer(&timer);
tEnd = sdkGetTimerValue(&timer);
printf(" execution time is %f ms \n", tEnd - tStart);
```

Alternatively, use the cuda event – based or omp_get_wtime() (openMP) time measurement

CUDA Runtime Initialization

CUDA runtime is initialized automatically whenever an CUDA runtime API function is called for the first time.

Some runtime parameters can be controlled manually, for example:

```
int gpuId = 7;
cudaSetDevice(gpuId);
```

Caveat: the initialization time might get included

in the measured execution time!

Solution: run the kernel for the first time without measuring the execution time to “warm up” the runtime environment. Then, run the kernel and measure the execution time. Better yet, run the kernel for a large number of times and then divide the total execution time by the number of runs.